

1. Contexto DevOps e o Conceito CALMS

O DevOps busca integrar as áreas de desenvolvimento (Dev) e operações (Ops) para aumentar a competitividade e eficiência. A capacidade de uma empresa adotar esses processos é frequentemente medida pela estrutura **CALMS**, que significa **Cultura, Automação, Lean, Medição e Compartilhamento (Sharing)**. Dentro dessa filosofia, o versionamento de código é visto como uma base essencial para a prática do processo, indo além de apenas salvar arquivos, mas compreendendo fluxos de criação e recuperação.

2. Fundamentos do Controle de Versão

O controle de versão é definido como uma prática essencial para gerenciar alterações em projetos de software, permitindo rastrear modificações, facilitar a colaboração e manter o histórico do código. Existem três modelos principais discutidos:

- **Sistemas Locais:** Mantêm um banco de dados de alterações no próprio computador do usuário.
- **Sistemas Centralizados (CVCS):** Utilizam um único servidor para armazenar todos os arquivos, facilitando o controle de acesso e backups, como o **Subversion** e **Perforce**.
- **Sistemas Distribuídos (DVCS):** Cada cliente possui uma cópia local completa do repositório, garantindo maior desempenho, autonomia (trabalho offline) e resistência a falhas, como o **Git**.

3. Git e GitHub

O **Git** é o sistema de controle de versão mais popular do mundo, desenvolvido por **Linus Torvalds** em 2005 para gerenciar o kernel do Linux. Já o **GitHub** é a plataforma de hospedagem de código mais conhecida, embora existam alternativas como Azure Repos e AWS CodeCommit.

- **Segurança e Boas Práticas:** O acesso ao GitHub deve ser protegido por **autenticação de dois fatores (2FA)**. Projetos bem estruturados devem conter um arquivo **README.md** (descrição, instalação, uso) e um arquivo **.gitignore** para evitar que arquivos desnecessários ou sensíveis sejam rastreados.
- **Protocolos de Conexão:** O acesso aos repositórios pode ser feito via **SSH** (usando chaves criptográficas, mais seguro para repositórios privados) ou **HTTP** (mais conveniente, mas que pode exigir login frequente).

4. Fluxo de Trabalho e Comandos Principais

O domínio do Git exige o entendimento de comandos fundamentais para o fluxo de trabalho:

- **Inicialização e Cópia:** `git init` (cria repositório local) e `git clone` (copia projeto remoto).
- **Ciclo de Alteração:** `git add` (prepara as mudanças no "Index") e `git commit` (salva um instantâneo/snapshot no histórico local).
- **Sincronização:** `git push` (envia para o servidor) e `git pull` (traz atualizações da equipe).
- **Colaboração Avançada:** O uso de **Branches** permite criar linhas separadas de desenvolvimento; o **Merging** combina essas linhas; e os **Pull Requests** fornecem uma estrutura para revisão de código antes da integração final.

5. Ferramentas de Apoio

Além da linha de comando, existem interfaces gráficas (GUIs) que simplificam o trabalho, como **SourceTree**, **GitKraken** e **TortoiseGit**. IDEs como o **Visual Studio Code** e **Eclipse** também possuem integração nativa para gerenciar o controle de versão diretamente no ambiente de desenvolvimento.

6. Infraestrutura como Código (IaC) e Gestão de Backup

No contexto DevOps, a gestão da infraestrutura e dos dados também é automatizada:

- **IaC:** Trata a infraestrutura como software programável, utilizando arquivos de configuração (ex: **Terraform**) para garantir ambientes idênticos e consistentes.
- **Backup em DevOps:** A automação de backups reduz erros humanos e garante a execução previsível conforme políticas de retenção. Ferramentas citadas incluem **Veeam**, **AWS Backup**, **Azure Backup** e o próprio **GitLab**, que oferece recursos para versionamento e cópia de segurança de repositórios.